

Projeto dados-pessoais

Questões — Etapa 1

1. O que significa `@RestController`? Qual a diferença entre `@RestController` e `@Controller`? Por que usamos o primeiro neste projeto?

`@RestController` é a junção de `@Controller` com `@ResponseBody`. Com `@Controller` puro, o retorno do método é interpretado como o nome de uma página HTML a ser renderizada. Com `@RestController`, o valor retornado é convertido diretamente para o corpo da resposta (JSON). Usamos `@RestController` porque este projeto é uma API REST, e não uma aplicação que renderiza páginas no servidor.

2. Transcreva o trecho de código do `PessoaController` responsável pela injeção de dependência. Explique o que é injeção de dependência e por que a injeção pelo construtor é a forma recomendada no Spring.

```
private final PessoaService pessoaService;

public PessoaController(PessoaService pessoaService) {
    this.pessoaService = pessoaService;
}
```

Injeção de dependência é quando o Spring cria e fornece os objetos que uma classe precisa, em vez dela mesma criá-los com 'new'. A injeção pelo construtor é a forma recomendada porque permite que o campo seja final (imutável) e deixa explícito, na assinatura da classe, tudo o que ela precisa para funcionar.

3. Para que serve o `Optional<Pessoa>` no retorno de `obterPessoa()`? O que aconteceria se o método retornasse `Pessoa` diretamente (sem `Optional`) e a pessoa não fosse encontrada?

O `Optional<Pessoa>` deixa explícito que a busca pode não encontrar nenhum resultado, evitando retornar null sem aviso. Se o método retornasse `Pessoa` diretamente e a pessoa não fosse encontrada, o retorno seria null, e qualquer tentativa de usar esse objeto (por exemplo, `pessoa.getNome()`) lançaria um `NullPointerException`.

4. O que é o `@PostConstruct` e quando ele é executado? O que aconteceria se tentássemos popular os dados diretamente na declaração do Map (fora de qualquer método)?

`@PostConstruct` marca um método que é executado uma única vez, logo após o Spring criar o bean e antes de qualquer requisição ser processada. Aqui ele é usado para popular os dados iniciais no Map. Se tentássemos popular o Map direto na declaração do campo, não funcionaria bem, pois essa inicialização depende de lógica executável (como o `contador.incrementAndGet()`), que só pode rodar dentro de um método, e não é o padrão esperado para lógica de inicialização gerenciada pelo Spring.

5. Quais anotações do `PessoaController` participam da composição da URL de acesso a um recurso? Explique como elas se combinam e dê um exemplo com a URL completa resultante para buscar uma pessoa pelo username.

As anotações são `@RequestMapping("/pessoas")`, na classe, e `@GetMapping("/{username}")`, no método. Elas se combinam somando os caminhos: URL final = `/pessoas/{username}`. Exemplo: `http://localhost:8080/pessoas/fulano`.

6. O método obterPessoas() retorna List<Pessoa>, mas o cliente recebe JSON. Como essa conversão acontece? Identifique qual biblioteca realiza a serialização, qual anotação do controller a ativa e por que a classe Pessoa precisa de um construtor sem parâmetros para que o processo funcione corretamente.

Quem realiza a serialização é a biblioteca Jackson, ativada pela anotação @RestController (por meio do @ResponseBody que ela incorpora). A classe Pessoa precisa de um construtor sem parâmetros porque o Jackson usa reflection para instanciar o objeto vazio primeiro e depois preencher os campos por meio dos setters durante a desserialização; sem esse construtor, o Jackson não consegue criar a instância.

7. O Map usa o username como chave. Que vantagem isso traz na operação de busca por username em comparação com armazenar as pessoas em uma List?

Buscar em um Map pela chave é praticamente instantâneo (complexidade $O(1)$), pois o acesso é feito diretamente pela posição calculada a partir do hash da chave. Em uma List seria necessário percorrer item por item até encontrar o valor (complexidade $O(n)$), o que se torna mais lento conforme a lista cresce.

8. O que acontece quando o navegador acessa GET /pessoas/xyz e a pessoa não é encontrada? Descreva o caminho do código desde a requisição até a resposta HTTP, identificando qual classe e método lança o erro e qual código de status é retornado.

O PessoaController.obterPessoa() chama pessoaService.obterPessoa("xyz"), que retorna um Optional vazio. O controller verifica optPessoa.isEmpty() e lança new ResponseStatusException(HttpStatus.NOT_FOUND), que o Spring converte automaticamente em uma resposta HTTP com código de status 404.